

Wetterstation ESP32

Dokumentation der Entwicklung einer Wetterstation mit dem ESP32
Teil 1

Eingereicht am: 03. März 2026

Dieses Dokument wurde mit L^AT_EX erstellt.

Autor: Anthony Timmel

Klasse: Fi24

Lernfeld: 7

Lernbereich: Cyberphysische Systeme

Contents

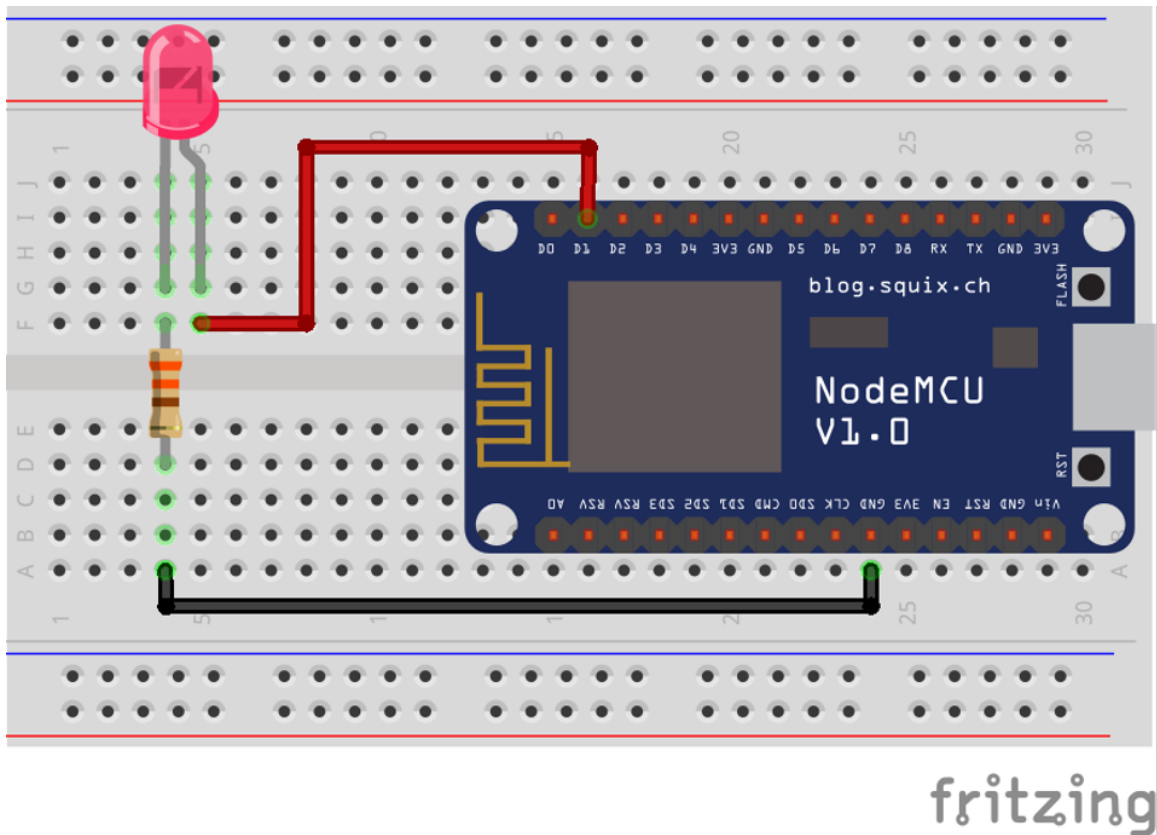
1	Definition der Schnittstellen	2
1.1	GPIO	2
1.2	UART	3
1.3	I2C	4
1.4	SPI	4
2	Aufbau des ESP32	5
3	Aufgabe Teil 1	8
3.1	Anforderungen	8
3.2	Hardware	8
3.2.1	Bauelemente	8
3.2.2	Vernetzung der Bauelemente	8
3.3	Programmcode	9
4	Aufgabe Teil 2	11
4.1	Anforderungen	11
4.1.1	Teil 1	11
4.1.2	Teil 2	11
4.2	Hardware	11
4.2.1	Bauelemente	11
4.2.2	Vernetzung der Bauteile	12
4.3	Programmcode	13
4.3.1	Programmcode für AM312	13
4.3.2	Programmcode für LM393	14
4.3.3	Überarbeitung des while-loops	14
5	Aufgabe Teil 3	17
5.1	Anforderungen	17
5.2	Hardware	17
5.2.1	Bauelemente	17
5.2.2	Vernetzung der Bauteile	17
5.3	Programmcode	17

1 Definition der Schnittstellen

1.1 Der Port GPIO

GPIO steht für **General Purpose Input/Output**. Dabei kann dieser entweder ein Eingang- oder Ausgangssignal repräsentieren. Es wird dazu genutzt, um mit weiteren einfachen Komponenten zu kommunizieren oder Steuern. In dieser Grafik wird der Aufbau mit einer LED & dem GPIO Port

Figure 1: GPIO Aufbau

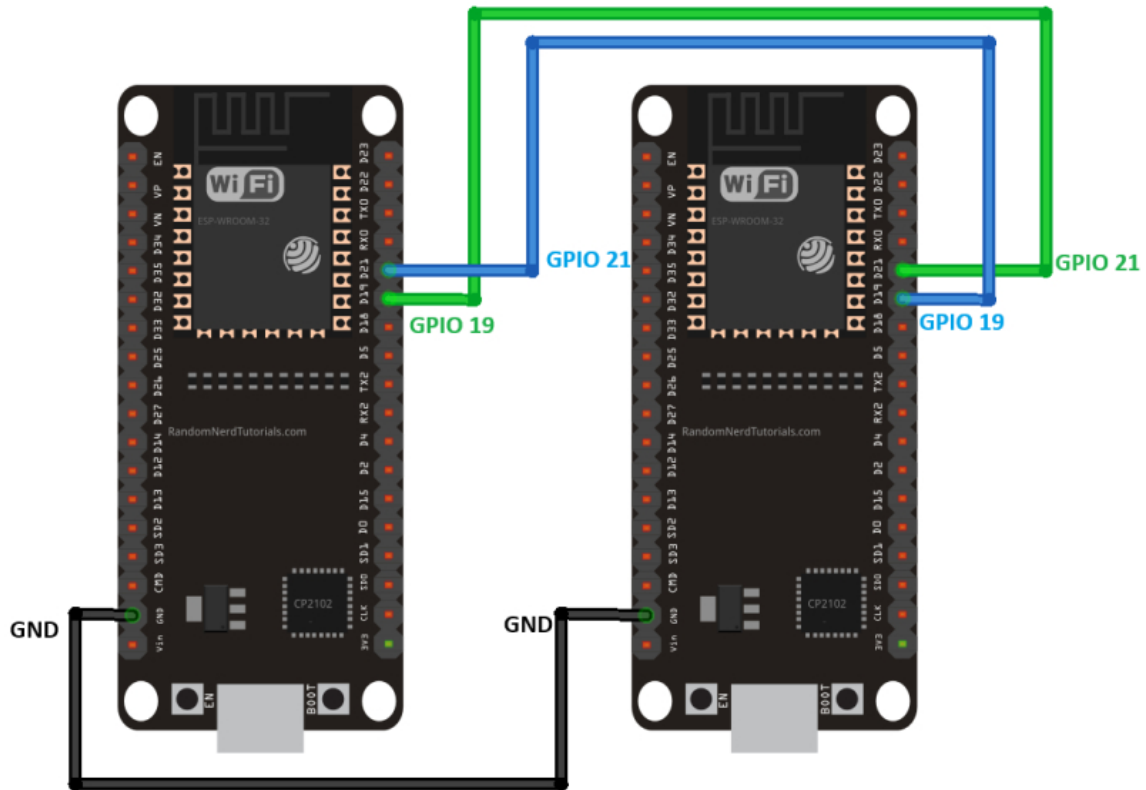


beschrieben. Die LED kann dann per Anweisung ein Stromsignal erhalten oder nicht. Dabei kann ein GPIO Port ausschließlich unter Spannung stehen, oder nicht.

1.2 Der Port UART

UART steht für **Universal Asynchronous Receiver/Transmitter**. Es übersetzt Daten zwischen der seriellen & paralleler Form.

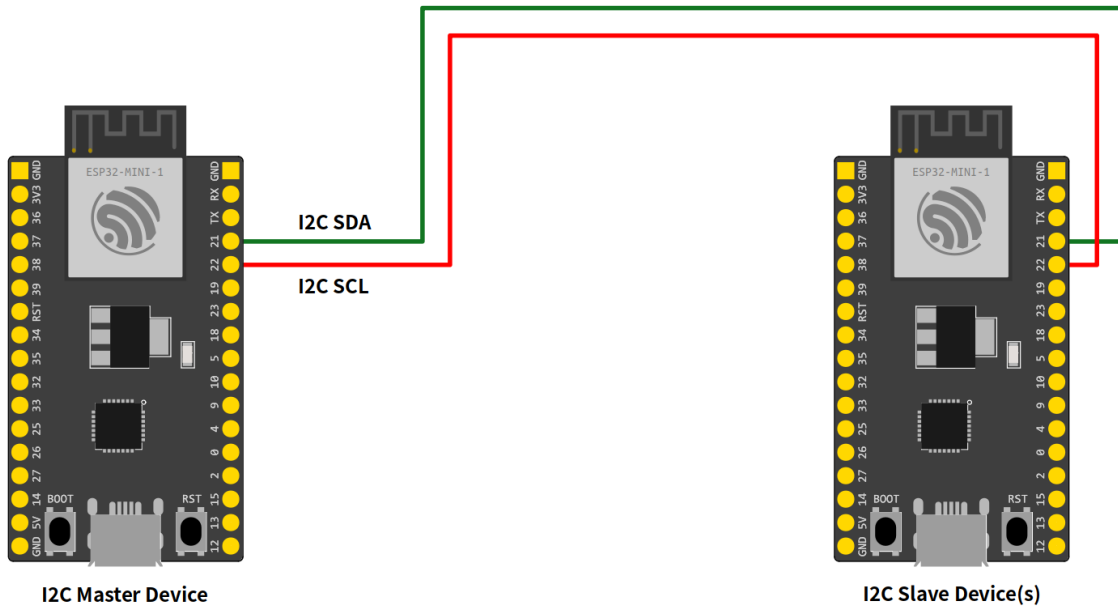
Figure 2: UART Aufbau



1.3 Der Port I2C

I2C oder auch **Inter-Integrated-Circuit** ist ein serieller Kommunikationsbus, der 2 spezielle GPIO Pins benötigt **SDA**(Serial Data Line) & **SCL**(Serial Clock Line). Dabei wird *I2C* benötigt, um Daten schnell & in großer Form zu übertragen, sowie die Uhrzeit synchron über Komponenten hinweg zu halten.

Figure 3: I2C Aufbau



1.4 Der Port SPI

SPI steht für **Serial Peripheral Interface** und ist ein synchrones serielles Datenprotokoll, das von Mikrocontrollern zur Kommunikation mit einem oder mehreren Peripheriegeräten verwendet wird. Beispielsweise kommuniziert der ESP32-Board mit einem Sensor, der SPI unterstützt, oder mit einem anderen Mikrocontroller.

Figure 4: SPI Aufbau



2 Aufbau des ESP32

Figure 5: ESP32 Internal Structure

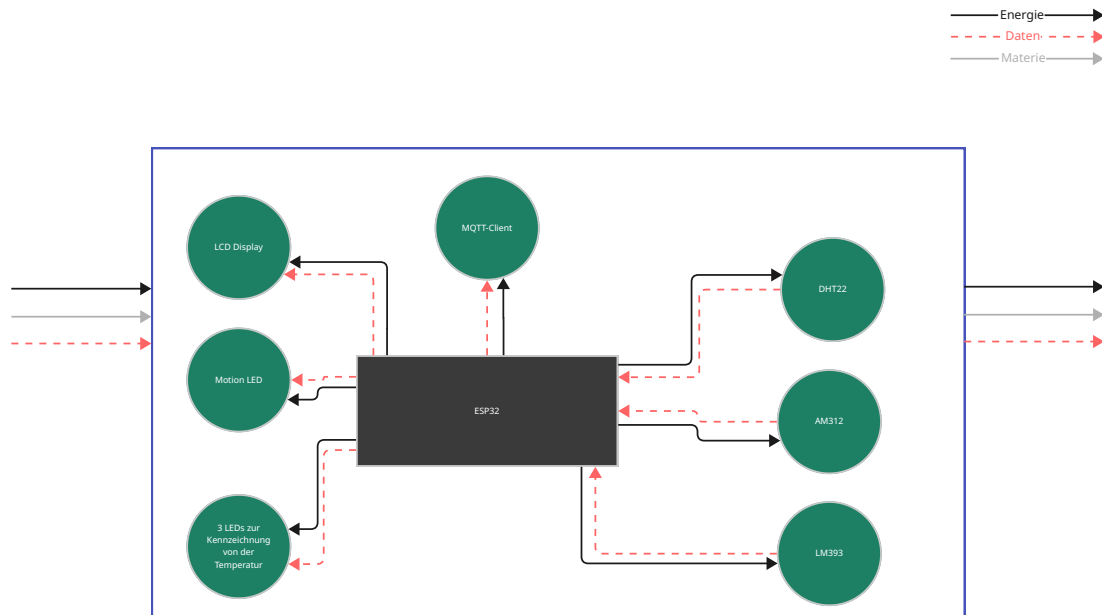


Figure 6: ESP32 Topansicht

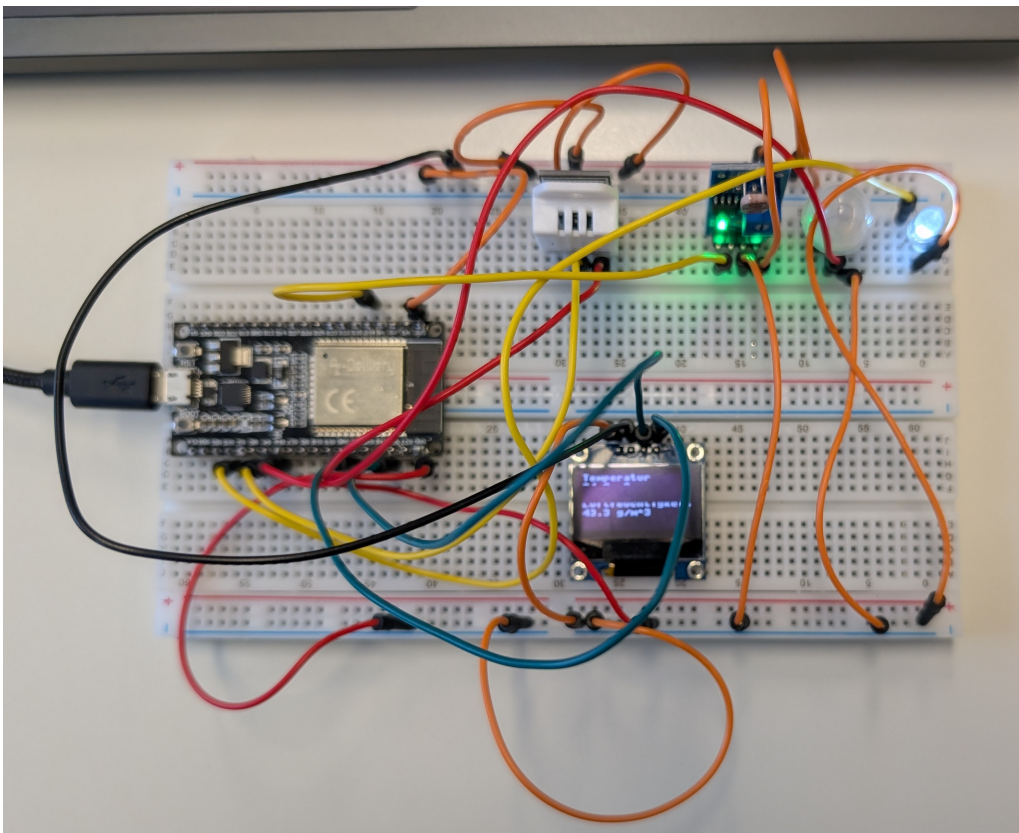
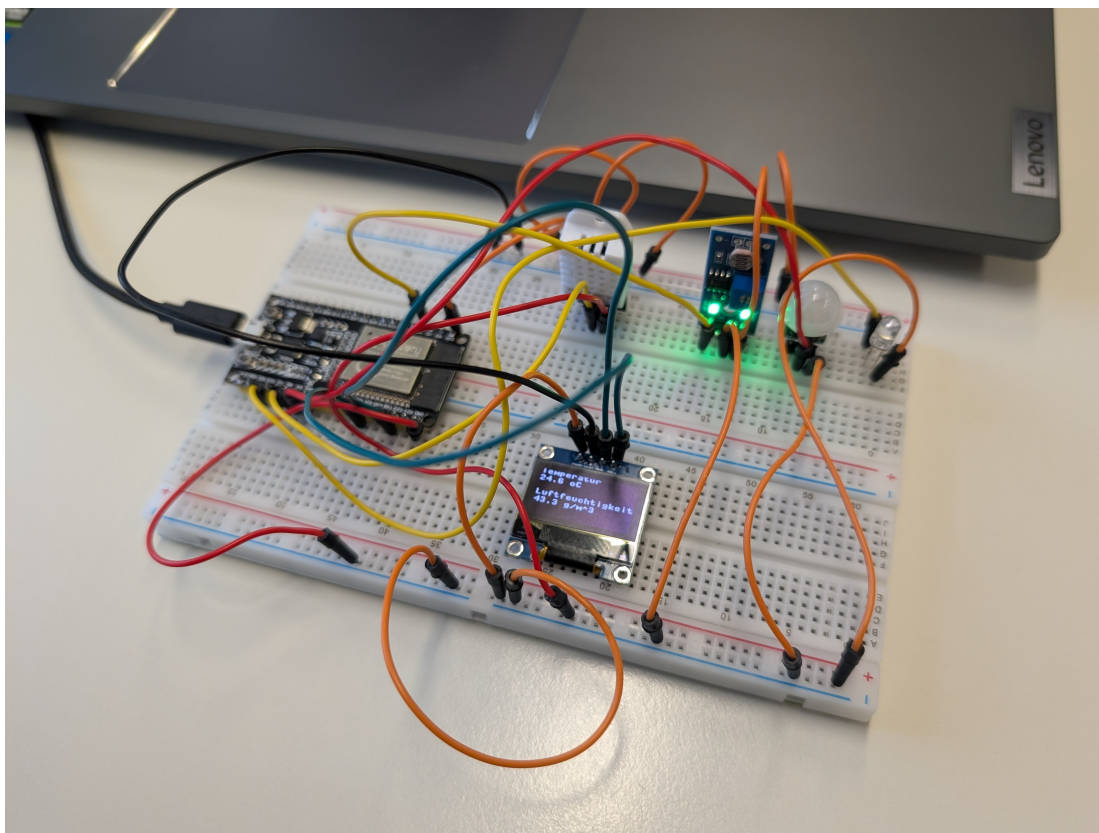


Figure 7: ESP32 Seitenansicht



3 Aufgabe Teil 1

3.1 Anforderungen

Bevor das cyber-physische System (CPS) in das **interne** Netzwerk integriert werden kann sollen sie testen, ob die Möglichkeit besteht mit einem ESP32 als Grundlage des CPS, die Temperatur und die Luftfeuchtigkeit mit einem Sensor (**DHT22**) einzulesen und zu verarbeiten. Für die sekundliche Übermittlung der Daten an den später auszuwertenden MQTT-Client sollen die Sensorwerte in das Datenformat JSON abgelegt werden.

Zusatz: Greifen Sie auf den Inhalt des in MicroPython erstellten JSON-Strings zu und **geben** Sie die Sensordaten mit der `print()`-Funktion im Terminal aus.

3.2 Die Vernetzung der Hardware auf dem Steckbrett

3.2.1 Bauelemente

ESP32

Der *ESP32* befindet sich bereits **vorinstalliert** auf dem Steckbrett und kann, durch das **Hinzufügen** des Stromversorgungs- & Datenübertragungskabels lässt sich die Einheit, von dem Computer aus Steuern.

Verteilung der Pin-Nutzung auf dem ESP32

Pin	Aufgabe
3V3	Stromversorgung des Bauteils
GND	Die Masse des ESP32
GPIO4	Universeller Eingangspin für die Messdaten es DHT22

DHT22

Das Erweiterungsmodul *DHT22* ist ein Modul, was es **ermöglicht**, die Raumtemperatur sowie die Luftfeuchtigkeit in der Umgebung **festzustellen**.

Verteilung der Pin-Nutzung auf dem DHT22

Pin	Aufgabe
VCC	Stromversorgung des Bauteils
GROUND	Die Masse des DHT22
DATA	Datenübertragungsport für die Messdaten es Bauteils

3.2.2 Vernetzung der Bauelemente

Der *ESP32* ist mit dem Pin **ADC2_0 GPIO4** mit dem *DHT22* **DATA** Pin verbunden. Die Stromversorgung des *DHT22* geht über den **VCC** Pin des Bauteils, dieser ist mit der Stromversorgung von dem **3V3** Pin des *ESP32* verbunden. Der **GROUND** Pin des Bauteils, ist mit der Masse **GND** des *ESP32* zusammengeschlossen, damit die Schaltung elektrotechnisch Funktionstüchtig werden kann.

3.3 Programmcode für den ESP32

```
def get_device_temp_and_humid():
    dht_device = DHT22(Pin(4, Pin.IN))
    dht_device.measure()
    temp = dht_device.temperature()
    humid = dht_device.humidity()

    return temp, humid
```

Mithilfe dieser Funktion, ist es uns möglich, die **Temperatur** und **Luftfeuchtigkeit** an dem **aktuellen Ort** zu bestimmen. Dafür sendet das Modul *DHT22* seine Daten an den Pin **G4**, diesen aben wir zuvor ([Pinverteilung ESP32](#)) definiert.

Durch **DHT22** aus der Datei **dht** erhalten wir die Möglichkeit, Daten über eine Erweiterung des Protokolls abzugreifen. Dafür haben wir den Input-Pin der DHT22 Klasse **DHT22(Pin(4, Pin.IN))** bei der Initialisierung **übergeben**. Im Anschluss ist es uns nun möglich, eine Messung **mithilfe** von **dht_device.measure()** können wir das Modul **auffordern**, eine Messung der Temperatur sowie Luftfeuchtigkeit vorzunehmen. Im Anschluss **erhalten** wir die Temperatur sowie Luftfeuchtigkeit über die jeweilige Methode des Objektes und geben diese am Ende der Funktion als **Pair** zurück.

```
while True:
    temp, humid = get_device_temp_and_humid()

    raw_json_object = {
        "Temperatur": temp,
        "Luftfeuchtigkeit": humid
    }

    json_object = json.dumps(raw_json_object)

    print(json_object)
    sleep(1)
```

Mithilfe **dieser** Schleife, ermöglicht es uns, bestimmte Bauteile **jede** Sekunde, auf dem *ESP32* abzufragen und im Anschluss, die Daten weiter zu verarbeiten. Dafür holen wir uns die **Temperatur** & **Luftfeuchtigkeit** über die Funktion **get_device_temp_and_humid()**. Im Anschluss erstellen wir ein Dictionary, um die Daten später im Programmcode leichter zu einem JSON-String zu konvertieren. Die Konvertierung zu einem JSON-String findet mit der Hilfe von **json** aus der Datei **json**. Dadurch, dass wir **bereits** einen Dictionary erstellt haben, können wir dies nun direkt der **dump** Funktion **übergeben**, und diese gibt uns einen vollständigen JSON-String zurück. Daraufhin geben wir diesen JSON-String in der Console aus und warten 1 Sekunde, bis die Schleife erneut durchläuft.

Der vollständige Programmcode für die Aufgabe Teil 1 sieht also wie folgt aus:

```
from machine import Pin
from dht import DHT22
from time import sleep
import json

def get_device_temp_and_humid():
    dht_device = DHT22(Pin(4, Pin.IN))
    dht_device.measure()
    temp = dht_device.temperature()
    humid = dht_device.humidity()

    return temp, humid

while True:
    temp, humid = get_device_temp_and_humid()

    raw_json_object = {
        "Temperatur": temp,
        "Luftfeuchtigkeit": humid
    }

    json_object = json.dumps(raw_json_object)

    print(json_object)
    sleep(1)
```

4 Aufgabe Teil 2

4.1 Anforderungen

4.1.1 Teil 1

Erweitern Sie das ESP32 um **einen** Bewegungsmelder und einen Fotowiderstand, dessen Daten Sie ebenfalls zu dem **bereits erstellten** JSON-Syntax hinzufügen. Das Ansprechen des Bewegungsmelders **soll** hierbei über eine LED signalisiert werden.

4.1.2 Teil 2

Ergänzen Sie Ihren Prototypen dahingehend, dass eine **grüne**, **gelbe** oder **rote** LED leuchten soll wenn die Temperatur $\geq 30\text{ °C}$ (rot), $\geq 25\text{ °C}$ und $< 30\text{ °C}$ (gelb) oder $\leq 20\text{ °C}$ (grün) beträgt

4.2 Hardware

4.2.1 Bauelemente

Inkrementeller Aufbau zu der [Teilaufgabe 1](#).

ESP32

Verteilung der Pin-Nutzung bei dem ESP32

Pin	Aufgabe
VCC	Stromversorgung des Bauteils
GND	Die Masse des LM393
GPIO34	Universeller Analoger Eingangspin des ESP32 für den LM393
GPIO16	Universeller Eingangspin des ESP32 für den AM312

AM312

Das Erweiterungsmodul *AM312* ist ein Modul, was zur **Erfassung** von **Bewegungen** verwendet wird.

Verteilung der Pin-Nutzung bei dem AM312

Pin	Aufgabe
VCC	Stromversorgung des Bauteils
GND	Die Masse des AM312
VOUT	Datenübertragungsport für die Messdaten es Bauteils

LM393

Das Erweiterungsmodul *LM393* ist ein Modul, was zur **Erfassung** der **Helligkeit** verwendet wird.

Verteilung der Pin-Nutzung bei dem AM312

Pin	Aufgabe
VCC	Stromversorgung des Bauteils
GND	Die Masse des LM393
AO	Analoger Datenübertragungsport für die Messdaten es Bauteils

4.2.2 Vernetzung der Bauteile

Der *ESP32* ist mit dem Pin **GPIO16** mit dem *AM312* **VOUT** Pin verbunden. Die Stromversorgung des *AM312* geht über den **VCC** Pin des Bauteils, dieser ist mit der Stromversorgung von dem **3V3** Pin des *ESP32* verbunden. Der **GND** Pin des Bauteils, ist mit der Masse **GND** des *ESP32* zusammengeschlossen, damit die Schaltung elektrotechnisch Funktionstüchtig werden kann.

Der *ESP32* ist mit dem Pin **ADC1_6 GPIO34** mit dem *LM393* **AO** Pin verbunden. Die Stromversorgung des *LM393* geht über den **VCC** Pin des Bauteils, dieser ist mit der Stromversorgung von dem **3V3** Pin des *ESP32* verbunden. Der **GND** Pin des Bauteils, ist mit der Masse **GND** des *ESP32* zusammengeschlossen, damit die Schaltung elektrotechnisch Funktionstüchtig werden kann.

4.3 Programmcode

4.3.1 Programmcode für AM312

```
def get_motion():  
    pir_pin = Pin(16, Pin.IN)  
    return bool(pir_pin.value())
```

Diese Funktion gibt den **aktuellen** Wert (**True/False**) zurück, ob der Bewegungsmelder *AM312*, **eine Bewegung** innerhalb von 100° zwischen 3-5 Metern **wahrgenommen** hat. Sollte **keine** Bewegung wahrgenommen sein, gibt dieser ein **False** zurück. Die Methode `pir_pin.value()` gibt jedoch eine Zahl (**1** oder **0**) zurück. Damit diese jedoch **besser Nachvollziehbar** im Programmcode ist, wird diese zu einem Boolean **gecastet**. Dadurch wird **sichergestellt**, dass die Erkennung von einer Bewegung **nur 2** unterschiedliche Zustände haben kann.

Für die LED definieren wir außerhalb einer Funktion folgendes:

```
led = Pin(15, Pin.OUT, drive=Pin.DRIVE_0)
```

Hierbei **definieren** wir den **GPIO15** Pin, als Ausgangspin mit einem **maximalen** Vorwiderstand, damit wir das LED Bauteil nicht beschädigen.

4.3.2 Programmcode für LM393

```
def get_luminance(gamma=0.7, r110_kohm=50, fixed_resistor_ohm=2000):  
  
    pin = ADC(Pin(34, Pin.IN))  
    pin.atten(ADC.ATTN_11DB)  
  
    V_REF = 3.3  
    MAX_ADC = 4095  
  
    raw_value = pin.read()  
  
    voltage_out = raw_value / MAX_ADC * V_REF  
  
    if voltage_out == 0:  
        return 0  
  
    resistance_ldr = fixed_resistor_ohm * (V_REF - voltage_out) / voltage_out  
  
    r110_ohm = r110_kohm * 1000  
  
    basis = (r110_ohm * pow(10, gamma) / resistance_ldr)  
    exponent = 1 / gamma  
  
    return pow(basis, exponent)
```

Zuerst richten wir den Pin für den *LM393* ein, indem wir die spezielle Klasse **ADC** dafür importieren und die Pinzuweisung dieser Klasse, im Constructor direkt übergeben. Im nächsten Schritt, erweitern wir den Messbereich des Analog-Digital-Converters (*ADC*) auf **3.3V** durch die Konstante **ADC.ATTN_11DB**. Sollte diese Einstellung nicht vorgenommen werden, wäre die *ADC* Spannung bis ca. **1.0V** präzise messbar.

Im Anschluss definieren wir 2 Konstanten für die Widerstandsberechnung & Umrechnung der Spannung (Volt).

V_REF ist hierbei die Referenzspannung von 3.3V, die das Erweiterungsmodul von dem ESP32 als Stromversorgung erhält.

MAX_ADC ist die Auflösung des analogen Signals. Dies wird hierbei auf **12-Bit** Auslösung gesetzt ($2^{12} - 1$).

raw_value erhält nun die analogen Daten des *LM393*.

voltage_out ist die korrekte Umrechnung des *12-Bit-Wertes* in Spannung (Volt). Bevor wir mit der Widerstandsrechnung anfangen dürfen, überprüfen wir, ob die Ausgehende Spannung **voltage_out** gleich 0 ist, wenn ja, geben wir direkt den Lux Wert 0 zurück, da bei einer ausgehenden Spannung von 0V kein Licht wahrgenommen werden kann, oder wird.

Nun erfolgt die Widerstandsrechnung ($R_{LDR} = \text{resistance_ldr}$), dabei ist die Formel wie folgt $R_{LDR} = R_{fix} * \frac{V_{REF} - V_{out}}{V_{out}}$.

Nach der Widerstandsrechnung, müssen wir, bevor wir die Lux berechnen können, die kOhm in Ohm umrechnen. Dafür rechnen wir die kOhm **r110_kohm** mit *1000 und erhalten die **r110_ohm**.

Jetzt ist der Weg frei, die Lux zu berechnen. Jedoch benötigen wir erst die Basis, das ist mit folgender Formel errechenbar. ($R_{110} * 10^{gamma} / R_{LDR}$)

Im Anschluss müssen wir den Exponenten berechnen, dies erfolgt über eine einfache Division $exponent = 1/gamma$.

Der letzte Schritt, berechnen wir nun die Lux, indem wir die **basis** hoch **exponent** rechnen, das Ergebnis geben wir dann zurück.

4.3.3 Überarbeitung des while-loops

Im Anschluss müssen wir den While-Loop mit den neuen Funktionen ergänzen. Die vollständige neue While-Schleife sieht nun wie folgt aus:

```

while True:
    temp, humid = get_device_temp_and_humid()
    motion = get_motion()

    luminance = get_luminance()

    if motion:
        led.value(1)
    else:
        led.value(0)

    raw_json_object = {
        "Temperatur": temp,
        "Luftfeuchtigkeit": humid,
        "Bewegung": motion,
        "Helligkeit": luminance
    }

    json_object = json.dumps(raw_json_object)

    print(json_object)
    sleep(1)

```

Wir haben die While-Schleife mit den `motion` & `luminance` variable aus den beiden bereits **vordefinierten** Funktionen erweitert. Damit jedoch nun, wie gefordert, auch die LED sich **updated**, sollte jemand **in den Bereich** des Sensor *AM321* kommen, muss die variable `motion` nun in einer `if-else` Bedingung **modifiziert** werden. Dafür setzen wir den LED-Pin auf 1 sollte Bewegung **wahrgenommen** werden, trifft dies **nicht** mehr zu, setzen wir 0, dadurch leuchtet die LED nicht mehr. Der Effekt bei `value()` ist, dass der Pin *GPIO16* entweder mit Spannung versorgt wird, oder nicht.

Der gesamte Programmcode sieht nun wie folgt aus:

```

from machine import ADC, Pin
from dht import DHT22
from time import sleep
import json

led = Pin(15, Pin.OUT, drive=Pin.DRIVE_0)

def get_device_temp_and_humid():
    dht_device = DHT22(Pin(4, Pin.IN))
    dht_device.measure()
    temp = dht_device.temperature()
    humid = dht_device.humidity()

    return temp, humid

def get_luminance(gamma=0.7,
                  r110_kohm=50, fixed_resistor_ohm=2000):

    pin = ADC(Pin(34, Pin.IN))
    pin.atten(ADC.ATTN_11DB)

    V_REF = 3.3
    MAX_ADC = 4095

    raw_value = pin.read()

    voltage_out = raw_value / MAX_ADC * V_REF

    if voltage_out == 0:
        return 0

```



```

    resistance_ldr = fixed_resistor_ohm * (V_REF - voltage_out) / voltage_out

    r110_ohm = r110_kohm * 1000

    basis = (r110_ohm * pow(10, gamma) / resistance_ldr)
    exponent = 1 / gamma

    return pow(basis, exponent)

while True:
    temp, humid = get_device_temp_and_humid()
    motion = get_motion()

    luminance = get_luminance()

    if motion:
        led.value(1)
    else:
        led.value(0)

    raw_json_object = {
        "Temperatur": temp,
        "Luftfeuchtigkeit": humid,
        "Bewegung": motion,
        "Helligkeit": luminance
    }

    json_object = json.dumps(raw_json_object)

    print(json_object)
    sleep(1)

```

5 Aufgabe Teil 3

5.1 Anforderungen

Es soll die Möglichkeit bestehen die Temperatur und die Luftfeuchtigkeit vor Ort auf **einem OLED Display** (SSD1306) visualisieren zu können

5.2 Hardware

5.2.1 Bauelemente

Inkrementeller Aufbau zu der [Teilaufgabe 2](#).

ESP32

Verteilung der Pin-Nutzung bei dem ESP32

Pin	Aufgabe
VCC	Stromversorgung des Bauteils
GND	Die Masse des ESP32
GPIO21 (WIRE_SDA)	Universeller Ein-/Ausgangspin mit ARDUINO Fähigkeit für SDA
GPIO22 (WIRE_SCL)	Universeller Ein-/Ausgangspin mit ARDUINO Fähigkeit für SCL

AM312

Das Erweiterungsmodul *SSD1306* ist ein Modul, was zum **Anzeigen** von **Informationen** (LCD-Monitor) genutzt wird.

Verteilung der Pin-Nutzung bei dem SSD1306

Pin	Aufgabe
3.3V	Stromversorgung des Bauteils
GND	Die Masse des SSD1306
SDA	Serial-Dataline für die Datenübertragung zum LCD-Display
SCL	Serial-Clock für die In-Sync Datenübertragung

5.2.2 Vernetzung der Bauteile

Der *ESP32* ist mit dem Pin **GPIO21** mit dem *SSD1306* **SDA** Pin und der **GPIO22** mit dem *SSD1306* **SCL** Pin verbunden. Die Stromversorgung des *SSD1306* geht über den **3.3V** Pin des Bauteils, dieser ist mit der Stromversorgung von dem **3V3** Pin des *ESP32* verbunden. Der **GND** Pin des Bauteils, ist mit der Masse **GND** des *ESP32* zusammengeschlossen, damit die Schaltung elektrotechnisch Funktionstüchtig werden kann.

5.3 Programmcode

```
pin = SoftI2C(sda=Pin(21), scl=Pin(22))
display = ssd1306.SSD1306_I2C(128, 64, pin)
```

pin wird hier wie zuvor, mit der **Wrapper**-Klasse **SoftI2C** erzeugt. Dabei wird der *sda* und *scl* Pin definiert. Die instanzierte Klasse, kann nun **erneut** wieder in eine Wrapper-Klasse **SSD1306_I2C** übergeben. Dadurch erlangen wir die Möglichkeit, das LCD-Display *SSD1306* anzusprechen.

Die *SSD1306* Klasse ist **keine** vordefinierte Klasse aus einem der MicroPython Dateien, sondern diese ist **manuell** auf dem *ESP32* hinterlegt. Der Programmcode für **diese** Datei ist aufgrund der Größe **nicht** in das Dokument implementiert.

Der Programmcode zur Erweiterung der *SoftI2C*-Klasse findest du [hier \(Github\)](#).

[Tutorial \(Random Nerd Tutorials\)](#) zur Einrichtung der Erweiterungsklasse.

Nachdem die *SSD1306*-Klasse nun verfügbar ist, kann die Funktion **display_text()** erstellt werden.

```
def display_text(temp, humid):
    display.fill(0)
    display.text("Temperatur", 0, 0)
    display.text(str(temp) + " Grad", 0, 10)
    display.text("Luftfeuchtigkeit", 0, 30)
    display.text(str(humid) + " g/m^3", 0, 40)
    display.show()
```

Damit sich der Text, bei einem Update **nicht** überlappt, muss zuvor die Fläche **zurückgesetzt** werden. Das Zurücksetzen der Anzeigefläche passiert, indem wir eine schwarze Fläche **display.fill(0)** zeichnen. Im Anschluss schreiben wir die **Temperatur & Luftfeuchtigkeit**, mit einem Versatz, auf den LCD-Monitor und aktualisieren die Anzeige mit **display.show()**.

Der gesamte Programmcode sieht nun folgendermaßen aus:

```

from machine import ADC, Pin, SoftI2C
from dht import DHT22
from time import sleep
import json
import ssd1306

led = Pin(15, Pin.OUT, drive=Pin.DRIVE_0)
pin = SoftI2C(sda=Pin(21), scl=Pin(22))
display = ssd1306.SSD1306_I2C(128, 64, pin)

def get_device_temp_and_humid():
    dht_device = DHT22(Pin(4, Pin.IN))
    dht_device.measure()
    temp = dht_device.temperature()
    humid = dht_device.humidity()

    return temp, humid

def get_luminance(gamma=0.7,
                  rl10_kohm=50, fixed_resistor_ohm=2000):

    pin = ADC(Pin(34, Pin.IN))
    pin atten(ADC.ATTN_11DB)

    V_REF = 3.3
    MAX_ADC = 4095

    raw_value = pin.read()

    voltage_out = raw_value / MAX_ADC * V_REF

    if voltage_out == 0:
        return 0

    resistance_ldr = fixed_resistor_ohm * (V_REF - voltage_out) / voltage_out

    rl10_ohm = rl10_kohm * 1000

    basis = (rl10_ohm * pow(10, gamma) / resistance_ldr)
    exponent = 1 / gamma
    return pow(basis, exponent)

def get_motion():
    pir_pin = Pin(16, Pin.IN)
    return bool(pir_pin.value())

def display_text(temp, humid):
    display.fill(0) # Fix bug, where chars are printed on the top of the previous one
    display.text("Temperatur", 0, 0)
    display.text(str(temp) + " °C", 0, 10)
    display.text("Luftfeuchtigkeit", 0, 30)
    display.text(str(humid) + " g/m^3", 0, 40)
    display.show()

while True:
    temp, humid = get_device_temp_and_humid()
    motion = get_motion()

    luminance = get_luminance()

    display_text(temp, humid)

```

```
if motion:
    led.value(1)
else:
    led.value(0)

raw_json_object = {
    "Temperatur": temp,
    "Luftfeuchtigkeit": humid,
    "Bewegung": motion,
    "Helligkeit": luminance
}

json_object = json.dumps(raw_json_object)

print(json_object)
sleep(1)
```